

Reporte Previo — Estructura del EXAMEN_FINAL.md

Entrega individual antes del examen. Cada estudiante documenta su propio módulo. Los 3 casos deben estar documentados independientemente del caso que les toque el día de la defensa. Si no hay snippets, esa subsección es 0 — no se busca el código el día del examen.

El examen requiere que los 3 módulos del trío estén desplegados en AWS y comunicándose. Esta entrega cubre solo el tuyo.

Los nombres de archivos, clases y métodos indicados en cada sección son referenciales. Usar la ruta y nombre real del proyecto propio.

Estructura requerida

Sección 1 — Despliegue en AWS

1.1 URLs en producción - Mi módulo: `http://3.144.161.11:5074/` - Mi módulo (Swagger):

`http://3.144.161.11:5074/swagger/index.html` - Belen (módulo compras): `http://54.89.193.126:5085/swagger#/` - Henry (módulo inventario): `http://3.80.84.228:5143`

1.2 Variables de entorno (.env.example)

```
DB_USER=admin
DB_PASSWORD=secret_password
DB_NAME=fastinventorydb

ASPNETCORE_ENVIRONMENT=Development

DB_CONNECTION_STRING=Host=postgres-db;Database=fastinventorydb;Username=admin;Password=secret_password

# MODO DE EJECUCIÓN (Development / Production)

# INTEGRACIÓN DINÁMICA
# Opción A: Usar mi propio Inventario (Docker)
INVENTORY_API_URL=http://inventory-api:8080

# Opción B: Usar el Inventario de un compañero (Ejemplo)
# INVENTORY_API_URL=http://54.1.2.3:5143

# URLs para el Frontend (Web App)
VITE_INVENTORY_API_URL=http://localhost:5143
VITE_PURCHASES_API_URL=http://localhost:5085
VITE_SALES_API_URL=http://localhost:5074
```

1.3 Cómo simular caída de Inventario

```
# Para simular la caída del servicio de Inventario, detén el contenedor de la API de inventario en el servidor
docker compose stop inventory-api

# Para restablecer el servicio, vuelve a iniciar el contenedor:
docker compose start inventory-api
```

Sección 2 — Notificación de restock (SSE)

2.1 Endpoint SSE en Inventario `Src/Controllers/InventarioController.cs` · método `StreamRestockEvents` (en `Inventory.Api/Controllers/RestockEventsController.cs`)

```
[HttpGet]
public async Task Stream(string companyCen, CancellationToken ct)
{
    Response.Headers.Append("Content-Type", "text/event-stream");
    Response.Headers.Append("Cache-Control", "no-cache");
    Response.Headers.Append("Connection", "keep-alive");

    var clientChannel = Channel.CreateUnbounded<RestockEvent>();
    var subscriptionId = broadcaster.Subscribe(companyCen, clientChannel);

    try
    {
        while (!ct.IsCancellationRequested)
        {
            using var delayTokenSource = CancellationTokenSource.CreateLinkedTokenSource(ct);
            var delayTask = Task.Delay(15000, delayTokenSource.Token);
            var readTask = clientChannel.Reader.WaitToReadAsync(ct).AsTask();

            var completedTask = await Task.WhenAny(readTask, delayTask);
            if (completedTask == readTask)
            {
                delayTokenSource.Cancel();
                if (await readTask)
                {
                    while (clientChannel.Reader.TryRead(out var evento))
                    {
                        var json = JsonSerializer.Serialize(evento, new JsonSerializerOptions { PropertyNaming
                        await Response.WriteAsync($"data: {json}\n\n", ct);
                        await Response.Body.FlushAsync(ct);
                    }
                }
            }
            else
            {
                await Response.WriteAsync($"": keep-alive\n\n", ct);
                await Response.Body.FlushAsync(ct);
            }
        }
    }
    finally
    {
        broadcaster.Unsubscribe(subscriptionId);
    }
}
```

```
}
}
```

2.2 Canal en memoria (registro en DI) `Program.cs` · registro del `Channel<RestockEvent>` como singleton (en `Inventory.Api/Program.cs`)

```
builder.Services.AddSingleton<Inventory.Api.Infrastructure.RestockEventBroadcaster>();
```

2.3 Consumidor en Ventas (frontend) `src/components/Layout.tsx` (consumo de `EventSource` y manejo de eventos SSE)

```
let inventoryUrl = import.meta.env.VITE_INVENTORY_API_URL || 'http://localhost:5143';
if (inventoryUrl.endsWith('/')) {
  inventoryUrl = inventoryUrl.slice(0, -1);
}
const sseUrl = inventoryUrl.endsWith('/api/inventory')
  ? `${inventoryUrl}/companies/${companyCen}/restock-events`
  : `${inventoryUrl}/api/inventory/companies/${companyCen}/restock-events`;

const source = new EventSource(sseUrl);

source.onmessage = (event) => {
  try {
    const restock = JSON.parse(event.data);
    const id = Math.random().toString(36).substring(2, 9);
    const msg = `📦 Restock: Se agregaron ${restock.quantity} unidades de "${restock.productName}" (Código: ${id})`;

    setAlerts(prev => [...prev, { id, message: msg }]);

    setTimeout(() => {
      setAlerts(prev => prev.filter(a => a.id !== id));
    }, 6000);
  } catch (err) {
    console.error("Error parsing restock event", err);
  }
};
```

2.4 Captura: notificación visible en Ventas

RESTAURANTE

Restaurante El Sabor

Dashboard

Catálogo

Stock

Compras

PDV

Cocina / Bar

Cambiar empresa

← Volver

Orden #3BDF47DE

CONFIRMADA

Restock: Se agregaron 1 unidades de "Agua Mineral 600ml" (Código: PRO-00005) en la bodega. Nuevo Stock: 25.

Restock: Se agregaron 2 unidades de "Picarones" (Código: PRO-00006) en la bodega. Nuevo Stock: 2.

Restock: Se agregaron 1 unidades de "Lomo Saltado" (Código: PRO-00002) en la bodega. Nuevo Stock: 1.

✓ Compra confirmada. Stock actualizado.

Proveedor

Distribuidora Miraflores

13/6/2026, 10:51:49 a. m.

Detalle de compra

PRO-00005 - Agua Mineral 600ml	x 1
PRO-00006 - Picarones	x 2
PRO-00002 - Lomo Saltado	x 1

Sección 3 — Resiliencia con Polly

3.1 Política implementada `Sales.Api/Program.cs` (registro de HttpClient con políticas de Retry y Circuit Breaker de Polly)

```
var retryPolicy = HttpPolicyExtensions
    .HandleTransientHttpError()
    .WaitAndRetryAsync(3, retryAttempt => TimeSpan.FromSeconds(Math.Pow(2, retryAttempt)));

var circuitBreakerPolicy = HttpPolicyExtensions
    .HandleTransientHttpError()
    .CircuitBreakerAsync(5, TimeSpan.FromSeconds(30));

builder.Services.AddHttpClient<IInventoryCatalogClient, InventoryCatalogClient>(client =>
{
    client.BaseAddress = new Uri(inventoryBaseUrl);
})
.AddPolicyHandler(retryPolicy)
.AddPolicyHandler(circuitBreakerPolicy);
```

3.2 Dónde se aplica `Sales.Api/Controllers/CatalogController.cs` · método donde se llama a `IInventoryCatalogClient`

file:///C:/Users/Zerom/OneDrive/Documentos/GitHub/ISW-312-PROJ1/Reporte_Previo.html

5/9

```
[HttpGet("products")]
public async Task<IActionResult> GetProducts(
    string companyCen,
    [FromQuery] string? search,
    [FromQuery] string? categoryCen,
    [FromQuery] string? warehouseCen,
    [FromQuery] bool onlyAvailable = true,
    [FromQuery] int page = 1,
    [FromQuery] int pageSize = 20,
    CancellationToken cancellationToken = default)
{
    var products = await inventoryClient.GetSellableProductsAsync(companyCen, search, categoryCen, warehouseCe
    return products is null ? StatusCode(502, "Inventory API did not return catalog data.") : Ok(products);
}
```

3.3 Respuesta cuando Inventario no responde

```
{
  "status": 503,
  "title": "Servicio no disponible",
  "detail": "El servicio de inventario no esta disponible temporalmente. Intente nuevamente en unos momentos.",
  "instance": "/api/sales/companies/EMP-00001/catalog/products",
  "traceId": "0HN12345678"
}
```

3.4 Captura: comportamiento con Inventario caído

Curl

```
curl -X 'GET' \
  'http://3.144.161.11:5074/api/sales/companies/5dd6217b-a457-4e65-872e-8850e190bfcf/catalog/products?onlyAvailable=true&page=1&pageSize=20' \
  -H 'accept: */*'
```

Request URL

```
http://3.144.161.11:5074/api/sales/companies/5dd6217b-a457-4e65-872e-8850e190bfcf/catalog/products?onlyAvailable=true&page=1&pageSize=20
```

Server response

Code	Details
503 <i>Undocumented</i>	Error: Service Unavailable Response body <pre>{ "status": 503, "title": "Servicio no disponible", "detail": "El servicio de inventario no esta disponible temporalmente. Intente nuevamente en unos momentos.", "instance": "/api/sales/companies/5dd6217b-a457-4e65-872e-8850e190bfcf/catalog/products", "traceId": "0HMMEH3EUNWZA:00000004" }</pre> Response headers <pre>cache-control: no-cache,no-store content-type: application/problem+json date: Sat, 20 Jun 2026 05:41:42 GMT expires: -1 pragma: no-cache server: Kestrel transfer-encoding: chunked</pre>

Sección 4 — Historial y dashboard

4.1 Modelo de la tabla de ventas `Sales.Api/Domain/Entities/TicketItem.cs` · entidad de detalle del ticket que almacena el `UnitPrice` histórico de la venta

```
public sealed class TicketItem
{
    public int Id { get; set; }
    public Guid Cen { get; set; } = Guid.NewGuid();
    public int TicketId { get; set; }
    public Guid TicketCen { get; set; }
    public int ProductId { get; set; }
    public Guid ProductCen { get; set; }
    public decimal Quantity { get; set; } = 1;
    public decimal UnitPrice { get; set; }
    public string Status { get; set; } = "PENDING";
    public string? Notes { get; set; }
}
```

4.2 Cómo se guarda el precio en la transacción `Sales.Api/Controllers/TicketsController.cs` · método `AddItem` donde se guarda el precio unitario del producto en la transacción

```
var item = new TicketItem
{
    TicketId = ticket.Id,
    TicketCen = ticket.Cen,
    ProductCen = productCen,
    Quantity = quantity,
    UnitPrice = request.UnitPrice,
    Notes = request.Notes,
    Status = "PENDING"
};

Db.TicketItems.Add(item);
await Db.SaveChangesAsync();
```

4.3 Query del dashboard mensual `Sales.Api/Controllers/DashboardController.cs` · método `MonthlySales` para consultar ventas del mes actual y mes anterior

```
var now = DateTime.UtcNow;
var startOfCurrentMonth = new DateTime(now.Year, now.Month, 1, 0, 0, 0, DateTimeKind.Utc);
var endOfCurrentMonth = startOfCurrentMonth.AddMonths(1).AddTicks(-1);

var startOfPreviousMonth = startOfCurrentMonth.AddMonths(-1);
var endOfPreviousMonth = startOfCurrentMonth.AddTicks(-1);

var currentMonthTotal = await Db.Payments
    .Where(x => Db.Tickets.Any(t => t.Id == x.TicketId && t.CompanyCen == company.Cen) && x.CreatedAt >= start
    .SumAsync(x => (decimal?)x.Amount) ?? 0;

var previousMonthTotal = await Db.Payments
    .Where(x => Db.Tickets.Any(t => t.Id == x.TicketId && t.CompanyCen == company.Cen) && x.CreatedAt >= start
    .SumAsync(x => (decimal?)x.Amount) ?? 0;
```

4.4 Captura: dashboard mes actual vs. mes anterior

